CS985 Assignment Report

Spotify Classification Problem 2025

Team 101

Team member

1. Titirat Seeharach (Student number : 202580746)
2. Umaima Hussain (Student number : 202553804)
3. Surya Subramanian (Student number : 202553846)
4. Calum Murphy (Student number : 202488764)
5. Zen Gawai (Student number : 202566417)

1. Introduction

The goal of this project is to develop a machine learning model to classify songs into different genres based on their data features. The dataset consists of multiple attributes, including beats per minute (bpm) ,danceability (dnce) and others. The challenge is to accurately predict a song's genre using these numerical and categorical features.

2. Data preparing

Before training models, we prepared the dataset through the following steps:

1. **Reading and Inspecting Data**
   - Used pandas to load the dataset.
   - Used scikit library to import the model.

In [6]:
```
# Import necessary libraries
# pandas: For data manipulation and preprocessing.
# scikit-learn: For implementing machine learning models and preprocessin

import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
```

2. **Feature selection**

Based on the training data, we selected nine key numerical features based on their relevance to genre classification:

Feature	Reason for Selection
dur	According to online articles, classical and rock songs tend to be longer, while pop songs are shorter.
spch	High speechiness can indicate spoken genres like rap.
pop	It might links with certain genres, helping to predict what genre a song belongs to.
nrgy	Genres like EDM and rock are known for high energy, whereas jazz or classical are lower in energy.
acous	It can indicate genres like folk, country, or acoustic rock.
year	Some genres are more prominent in certain time periods.
dB	Loudness is shown in genres like rock (loud).
dnce	Danceability is often high in pop and electronic genres.
artist	We have to assume that each artist has their style associated with specific genres, like Taylor Swift, who has distinct music in 'pop.'

3. **Feature Scaling and Encoding**
   - Used `StandardScaler()` to normalize numerical features.
   - Used `LabelEncoder()` to convert categorical target variables (top genre) into numerical values for model compatibility.
   - Later, we introduced One-Hot Encoding for the artist feature, leading to improved model performance.

3. Data cleaning

We need to check the training data to prepare it for use.In this file, we found an **empty space** in the **'top genre'** column, so we decided to remove it to reduce errors when using machine learning methods.

1. We need to write code to read both the training file and test file using the pandas library after importing it.

In [ ]:
```
# Please check the file path before running code
# According to the dataset, we have two datasets to consider: new refined
train_df = pd.read_csv(r"C:\Users\user\Downloads\cs-985-6-spotify-classif
test_df = pd.read_csv(r"C:\Users\user\Downloads\cs-985-6-spotify-classifi

# If you have a code file  which is the same folder in tran and test file
# train_df= pd.read_csv("refinedTrainingData.csv")
# test_df = pd.read_csv("CS98XClassificationTest.csv")
```

In [ ]:
```
# Check column both train and test dataset
print(train_df.columns)
print(test_df.columns)
```

After we remove the empty spaces .We create a new file called **'refinedTrainingData.csv'**. This file will be used to train the model to find a way to predict music genres from the relationship between **'top genre'** and the selected features.

Other features have least connect about the music genre such as **ID** (just identify the name of music) and **title** (It doesn't provide direct information for genre prediction).

In our code, we have:

- **X_train** for the eight selected features in the train file (refinedTrainingData.csv).
- **y** for the **top genre** (the target variable).
- **X_test** for the same eight selected features in the test file (CS98XClassificationTest).

In [ ]:
```
# Preprocessing for training data
X_train = train_df[['dur', 'spch', 'pop','nrgy', 'acous', 'year', 'dB', '
y_train = train_df['top genre']  # Target variable for training

# Preprocessing for test data (without target variable)
X_test = test_df[['dur', 'spch', 'pop','nrgy', 'acous', 'year', 'dB', 'dn
```

4. Data analyzing (Only training data)

After removing missing data during the data cleaning process, we can recheck for outliers, examine the data structure, and assess the correlation in the training dataset

In [8]:
```
# Import the matplotlib and seaborn libraries for data visualization to b
import matplotlib.pyplot as plt
import seaborn as sns
```

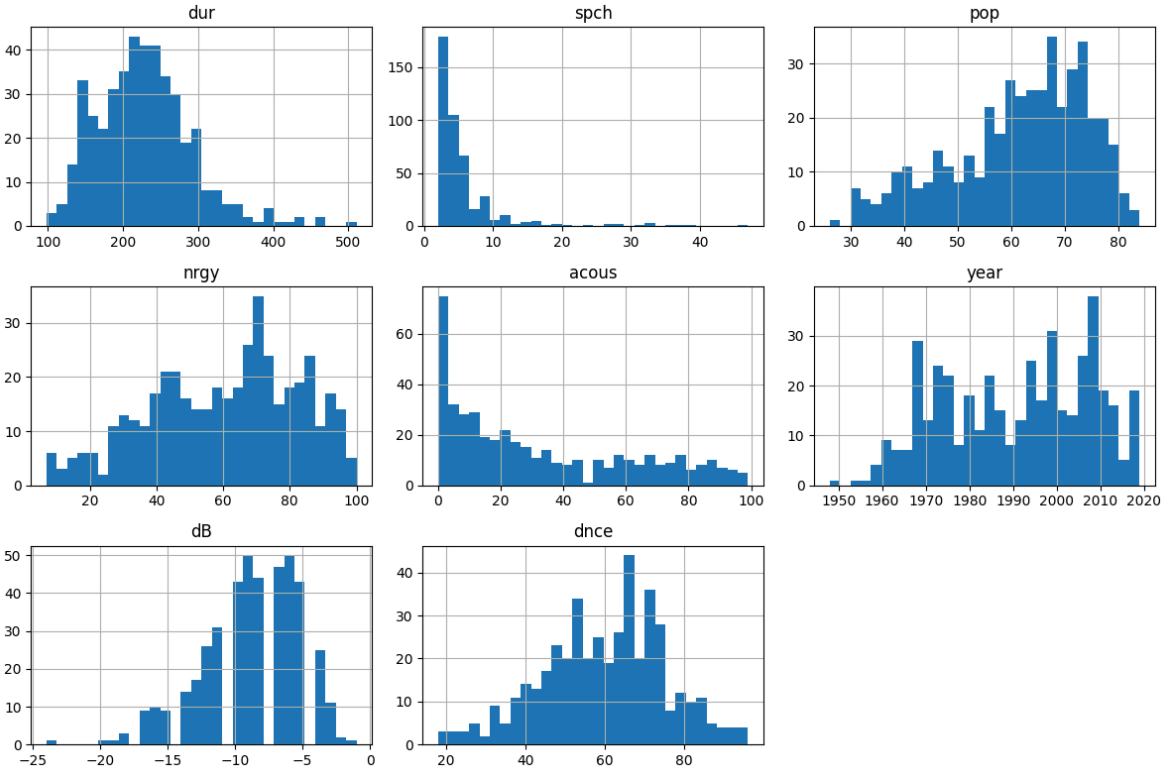
In [ ]:
```
# Check the first few rows of the dataset
print(train_df.head(10))
```

In [ ]:
```
# Summarize statistics data for numerical columns
print(train_df.describe())
```

In [ ]:
```
# Summarize data types of each column
print(train_df.dtypes)
```

Histograms plot - visualize the distribution of numerical data, allowing us to understand its shape

In [9]:
```
# Plot histograms for numerical features in the dataset
train_df[['dur', 'spch', 'pop', 'nrgy', 'acous', 'year', 'dB', 'dnce']].h
plt.tight_layout()
plt.show()
```



Checking outliers by Box plot - visualize how each category (e.g. artist) is related to the target variable.

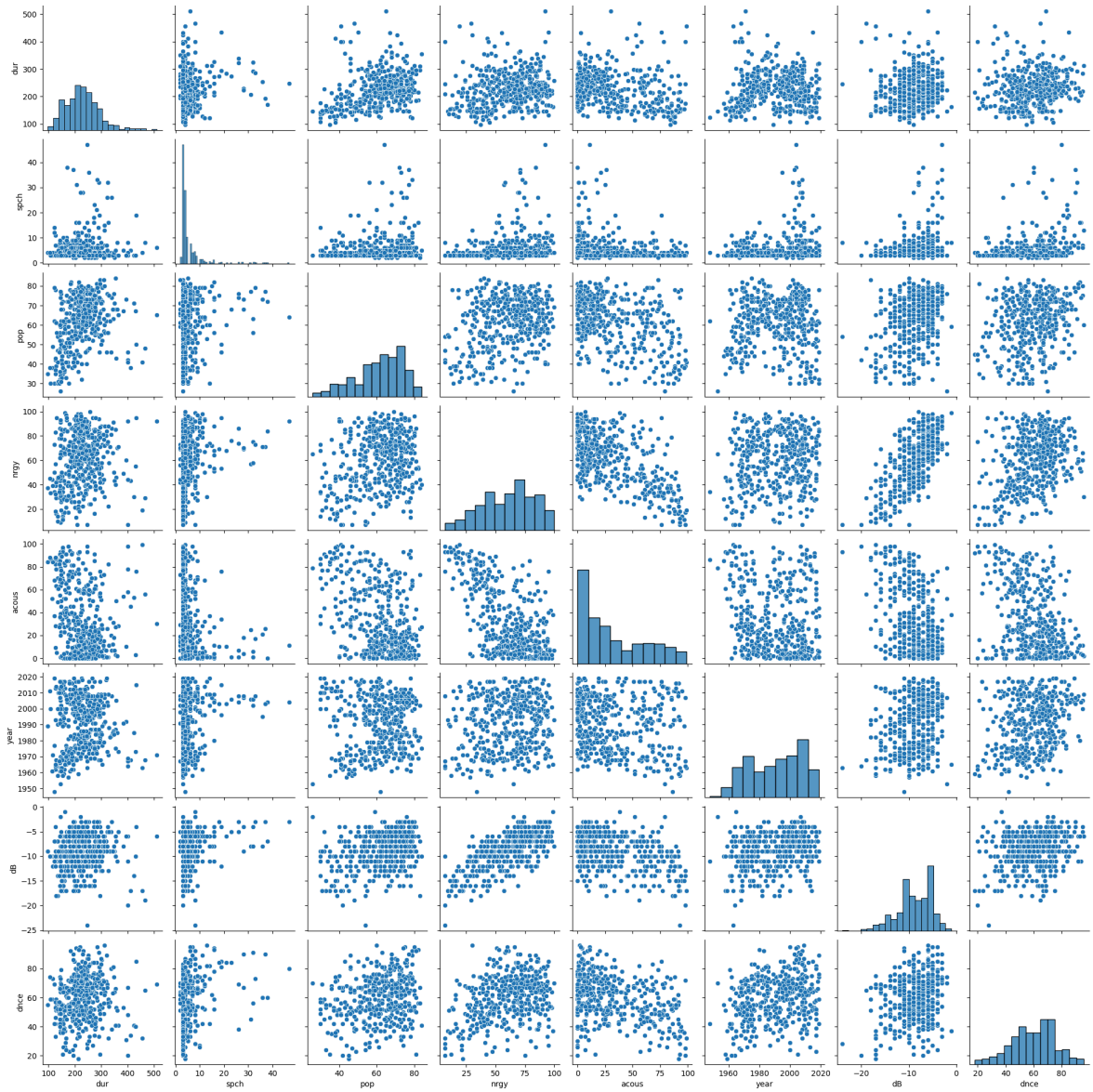
In [ ]:
```
# Box plot for checking outliers in the numerical columns
plt.figure(figsize=(12, 8))
sns.boxplot(x='top genre', y='dur', data=train_df)
plt.xticks(rotation=45)
plt.title('Duration by Top Genre')
plt.show()
```

Correlation heatmap -check the correlation between your input features , as highly correlated features can sometimes cause problems in model performance (especially in linear models).

In [ ]:
```
# Create correlation heatmap
corr_matrix = train_df[['dur', 'spch', 'pop', 'nrgy', 'acous', 'year', 'd
plt.figure(figsize=(10, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidt
plt.show()
```

Check how individual features correlate or separate the classes in the target variable.

In [ ]:
```
# Pair plot to look at relationships between features
sns.pairplot(train_df[['dur', 'spch', 'pop', 'nrgy', 'acous', 'year', 'dB
plt.show()
```



5. Methodology

After preparing the training data file, we can use various models to evaluate their performance scores.

For the classification, we have models such as:

1. Classification Models:
    - 1.1 Logistic regression: a simple baseline model suitable for linearly separable data.
    - 1.2 SVM for classification: good for both linear and non-linear classification tasks.
    - 1.3 Decision tree: allows interpretation of feature importance but prone to overfitting.
    - 1.4 Ensemble method(such as Stacking,Voting)
    - 1.5 One-hot encoder : improved categorical feature handling, particularly with the 'artist' column.

After that, we will create a table showing which model gets the performance score.

2. Choose one of the five methods (as described in step 1) and run the code (as shown in "Example code" in "Model Justification") to generate the new_test CSV file.
3. After we get the new_test CSV file, we have to remove all columns except for **'ID'** and **'Top genre'**.
4. Get the score by uploading the new_test CSV file to the **'Submit Prediction'** section on this website:"https://www.kaggle.com/competitions/cs-985-6-spotify-classification-problem-2025/leaderboard"

After training each model chosen in sections 1.1 to 1.5, we obtain the performance score for each model. We then present a performance score table showing the top four scores from each model.

Performance score

Model/Techniques	Accuracy	Observations
Logistic Regression	0.46428	Good baseline but struggled with non-linear relationships.
Stacking	0.58928	Best performance by combining models.
Voting	0.41071	Underperformed due to model similarity.
One-Hot Encoder	0.48124	Improved performance by encoding 'artist' without assumptions.

According to the table, the highest score is 0.58928, which we achieved using the **Stacking** model, whereas the **Voting model** received the lowest score of 0.41071. The One-Hot encoder and Logistic regression performed moderately with scores of 0.48124 and 0.46428, respectively.

6 .Model Justification

Stacking (Best Model: 58.9% Accuracy)

Performance & Observations	Reasons for Performance	Limitations	Improvements for the future
Achieved the highest accuracy among tested models.	- Combines diverse base models (Random Forest, Linear SVC, Logistic Regression).	- Higher computational cost compared to simpler models.	- Experiment with different meta-models (e.g., Gradient Boosting instead of Logistic Regression).

Logistic Regression (Initial Strong Performer: 46.4% Accuracy)

Performance & Observations	Reasons for Performance	Limitations	Improvements for the future
Strong baseline model with good accuracy	- Works well with linearly	- Cannot model complex, non-	- Introduce polynomial features

Performance & Observations	Reasons for Performance	Limitations	Improvements for the future
before more complex models were introduced.	separable data.	linear relationships.	to model non-linear relationships.

Voting Classifier (Underperformed: 41% Accuracy)

Performance & Observations	Reasons for Performance	Limitations	Improvements for the future
Expected to improve performance by aggregating predictions but performed worse than Stacking.	- Reduces variance by combining models.	- Logistic Regression, Random Forest, SVM were too similar, leading to redundant predictions.	- Assign higher weights to stronger models using weighted voting.

One-Hot Encoder (Moderate Performance: 48.1% Accuracy)

Performance & Observations	Reasons for Performance	Limitations	Improvements for the Future
Improved accuracy compared to basic Logistic Regression and Voting but did not outperform Stacking.	- Encodes categorical variables (e.g., 'artist') without making assumptions about relationships.	- High-dimensional data due to the large number of unique artists.	- Apply dimensionality reduction techniques (e.g., PCA) to reduce feature explosion.

Example of Logistic regression code

We started with Logistic Regression, encoding genres with LabelEncoder, then scaled, transformed, and trained the model with 1000 iterations for better performance.

In [ ]:
```
import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier

# Select Relevant Features
features = ["bpm", "nrgy", "dnce", "dB", "val", "acous", "spch", "dur",
target = "top genre"

# Drop rows with missing target values
train_df = train_df.dropna(subset=[target])

# Encode Categorical Features
# Label Encoding for 'top genre'
label_encoder = LabelEncoder()
train_df[target] = label_encoder.fit_transform(train_df[target])  # Enc

# Target Encoding for 'artist' (Convert artist to its mean genre encodir
artist_mean_genre = train_df.groupby("artist")[target].mean()
train_df["artist_encoded"] = train_df["artist"].map(artist_mean_genre)
```

```
test_df["artist_encoded"] = test_df["artist"].map(artist_mean_genre)

# Fill NaN values (for unseen artists in test set)
test_df["artist_encoded"].fillna(train_df[target].mean(), inplace=True)

# Normalize Numerical Features
scaler = StandardScaler()
numerical_features = ["bpm", "nrgy", "dnce", "dB", "val", "acous", "spch

train_df[numerical_features] = scaler.fit_transform(train_df[numerical_f
test_df[numerical_features] = scaler.transform(test_df[numerical_feature

# Train-Test Split
X_train = train_df[numerical_features]
y_train = train_df[target]
X_test = test_df[numerical_features]

# Train Random Forest Classifier
rf_model = RandomForestClassifier(n_estimators=200, random_state=42, max
rf_model.fit(X_train, y_train)

y_pred = rf_model.predict(X_test)

# Decode the numerical predictions back to genre names
y_pred_genres = label_encoder.inverse_transform(y_pred)

# Assign the decoded genres to the test DataFrame
test_df['top genre'] = y_pred_genres

# Save predictions with original test data
output_file = 'Logistic.csv'
test_df.to_csv(output_file, index=False)

# Display the predictions alongside the test data
print("\nSample predictions (first 10 rows):")
print(test_df[['title', 'artist', 'top genre']].head(10))
```

Example of Voting code

We then tried implementing different techniques and encoded features, normalized data, and trained a soft-voting ensemble (Random Forest[RF], Logistic Regression[LR], Support Vector Machine[SVM]) to predict genres, saving results to a CSV.

In [ ]:

```
# Import required libraries
import pandas as pd

from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC


# Use this data set for helping to convert "genre" from numerical value
train_file = "refinedTrainingData.csv"
test_file = "CS98XClassificationTest.csv"
train_df = pd.read_csv(train_file)
test_df = pd.read_csv(test_file)
```

```
# Select Relevant Features
# #features = ["bpm", "nrgy", "dnce", "dB", "val", "acous", "spch", "dur
# Updated Numerical Features List
numerical_features = [
    "bpm", "nrgy", "dnce", "dB", "val", "acous", "spch", "dur", "year",
]
target = "top genre"


# Encode Categorical Features
# Label Encoding for 'top genre'
label_encoder = LabelEncoder()
train_df[target] = label_encoder.fit_transform(train_df[target])  # Enco

# Target Encoding for 'artist' (Convert artist to its mean genre encodir
artist_mean_genre = train_df.groupby("artist")[target].mean()
train_df["artist_encoded"] = train_df["artist"].map(artist_mean_genre)
test_df["artist_encoded"] = test_df["artist"].map(artist_mean_genre)

# Fill NaN values (for unseen artists in test set)
test_df["artist_encoded"].fillna(train_df[target].mean(), inplace=True)


# Normalize Numerical Features
scaler = StandardScaler()
train_df[numerical_features] = scaler.fit_transform(train_df[numerical_f
test_df[numerical_features] = scaler.transform(test_df[numerical_feature

# Train-Test Split (Optional - Cross-validation could also be used)
X_train = train_df[numerical_features]
y_train = train_df[target]
X_test = test_df[numerical_features]

# Define Models for Ensemble

# Random Forest Classifier
# n_estimator choose for 200 trees, provided good accuracy while keeping
rf_model = RandomForestClassifier(n_estimators=200, random_state=42, max

# Logistic Regression
lr_model = LogisticRegression(max_iter=1000, random_state=42, class_weig

# SVM Classifier with probability enabled for soft voting
# use "rbf" which works better for non-linear relationships in these fea
# "probability = True" for allowing the model to provide probability sco
svm_model = SVC(kernel='rbf', probability=True, random_state=42, class_w

# Ensemble Model - Soft Voting
ensemble_model = VotingClassifier(
    estimators=[
        ('rf', rf_model),
        ('lr', lr_model),
        ('svm', svm_model)
    ],
    voting='soft'  # Soft Voting for probability averaging
)

# Train the Ensemble Model
```

```
ensemble_model.fit(X_train, y_train)

# Make Predictions on Test Data
y_pred = ensemble_model.predict(X_test)

# Decode the numerical predictions back to genre names
y_pred_genres = label_encoder.inverse_transform(y_pred)

# Assign the decoded genres to the test DataFrame
test_df['top genre'] = y_pred_genres

# Save predictions with original test data
output_file = 'rf_svm_lr.csv'
test_df.to_csv(output_file, index=False)

# Display the predictions alongside the test data
print("\nSample predictions (first 10 rows):")
print(test_df[['title', 'artist', 'top genre']].head(10))
```

Example of One hot encoder code

We then introduced one hot encoding to our model so that it can make it all more efficient and avoid any data assumptions by the model

In [ ]:

```
####### Encode Categorical Features

# Import required libraries (for run individual cell)
# import pandas as pd
# from sklearn.preprocessing import StandardScaler, LabelEncoder
# from sklearn.ensemble import RandomForestClassifier
# from sklearn.metrics import accuracy_score, classification_report

# Use this data set for helping to convert "genre" from numerical value
train_file = "refinedTrainingData.csv"
test_file = "CS98XClassificationTest.csv"
train_df = pd.read_csv(train_file)
test_df = pd.read_csv(test_file)

# Select Relevant Features
features = ["bpm", "nrgy", "dnce", "dB", "val", "acous", "spch", "dur",
target = "top genre"

# Drop rows with missing target values
train_df = train_df.dropna(subset=[target])

# Encode Categorical Features
# Label Encoding for 'top genre'
label_encoder = LabelEncoder()
train_df[target] = label_encoder.fit_transform(train_df[target])  # Enc

# One-Hot Encode 'artist' column
train_df = pd.get_dummies(train_df, columns=['artist'], drop_first=True)
test_df = pd.get_dummies(test_df, columns=['artist'], drop_first=True)

# Align columns in test set with train set (in case some artists are mis
test_df = test_df.reindex(columns=train_df.columns)  # Align columns
test_df.fillna(0, inplace=True)  # Fill NaN values with 0
```

```
# Updated Numerical Features List
numerical_features = [
    "bpm", "nrgy", "dnce", "dB", "val", "acous", "spch", "dur", "pop"
] + [col for col in train_df.columns if col.startswith('artist_')]  # Ad

# Normalize Numerical Features
scaler = StandardScaler()
train_df[numerical_features] = scaler.fit_transform(train_df[numerical_f
test_df[numerical_features] = scaler.transform(test_df[numerical_feature

# Prepare Train-Test Split
X_train = train_df[numerical_features]
y_train = train_df[target]
X_test = test_df[numerical_features]

# Train Random Forest Classifier
rf_model = RandomForestClassifier(n_estimators=200, random_state=42, max
rf_model.fit(X_train, y_train)

# Make Predictions on Test Data
y_pred = rf_model.predict(X_test)

# Decode the numerical predictions back to genre names
y_pred_genres = label_encoder.inverse_transform(y_pred)

# Assign the decoded genres to the test DataFrame
test_df['top genre'] = y_pred_genres

# Save predictions with original test data
output_file = 'one_artist_encoding.csv'
test_df.to_csv(output_file, index=False)

# Print sample predictions
print("\nTest Predictions (first few rows):")
print(test_df[["Id", "top genre", "title"]].head(10))
```

Final solution using Ensemble (Stacking) code

We then proceed to experiment with stacking as it will help us maximize the power of multiple models. We have configured and gone through multiple combinations and came through this which gave us the most stable result. During the experimentation we came across some issues like imbalanced data for which we used smote to oversample the minority classes and for better and cleaner data we used resampling and removed classes with 1 sample and for the model to perceive the input better we used one hot encoding on the "artist" column specifically as it is the only feature that is a string. We use class weight parameters for our models to keep everything balanced. After some trial and error we chose the following combination of models:

- **Random Forest:** Random forest became our strongest choice as an estimator because it is good with imbalanced data like in our case, reduces overfitting and gives more weightage to underrepresented classes. Helping with non-linearity and complexity in the data.
- **LinearSVC:** This model works on things that random forest doesn't focus on like linear relationships and we also added a standard scalar in this which it

uses to normalize the features.

- **Logistic Regression:** We are using this model as a meta model which would be the final aggregator for both of the previous model's predictions and this helps in avoiding overfitting by reducing bias.

Parameter changes values:

- n_estimaters = 200 // using 200 gives us a balance between complexity and computation
- random_state = 50 // this helps us keep a constant result specially in case of random forest model
- C = 0.1 // is used to regularize the model to prevent overfitting
- max_iter = 1000 // increasing the value from default 100 to 1000 helps the model converge more data incase of complex data

!!!! Make sure you get this library first

In [ ]:
```
# Make sure you have the imbalanced-learn library installed.
# If it's not installed yet, you can install it
# Using imbalanced-learn to handle class imbalance using SMOTE.
# install the xgboost library directly
### You can use the differnt way to install both library.
!pip install imbalanced-learn
!pip install xgboost
```

In [ ]:
```
##Stacking with randomforest and linearSVC(Scaled)
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
from xgboost import XGBClassifier
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
from sklearn.pipeline import make_pipeline

train_df = pd.read_csv("refinedTrainingData.csv")
test_df = pd.read_csv("CS98XClassificationTest.csv")

train_df_encoded = pd.get_dummies(train_df, columns=['artist'], drop_fir
test_df_encoded = pd.get_dummies(test_df, columns=['artist'], drop_first

test_df_encoded = test_df_encoded.reindex(columns=train_df_encoded.colum

X_train = train_df_encoded[['dur', 'spch', 'pop', 'nrgy', 'acous', 'year
y_train = train_df['top genre']

label_encoder_genre = LabelEncoder()
y_train_encoded = label_encoder_genre.fit_transform(y_train)

# Features for test data
X_test = test_df_encoded[['dur', 'spch', 'pop', 'nrgy', 'acous', 'year',

# Scale the features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)

# Stacking model with RandomForest and LinearSVC as base learners, Logis
estimators = [
    ('rf', RandomForestClassifier(n_estimators=200, random_state=50, cla
    ('svr', make_pipeline(StandardScaler(), LinearSVC(C=0.01,random_stat
]

# StackingClassifier with LogisticRegression as the final estimator
clf = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(max_iter=1000)
)
rare_classes = [cls for cls, count in zip(*np.unique(y_train_encoded, re

# Filter out rare classes
mask = ~np.isin(y_train_encoded, rare_classes)
X_train_filtered = X_train[mask]
y_train_filtered = y_train_encoded[mask]

smote = SMOTE(k_neighbors=1,random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_filter

# Train the stacking model
clf.fit(X_train_resampled, y_train_resampled)

# Predict the genres for the test set
y_pred = clf.predict(X_test)

# Inverse transform the predicted labels back to the original genre labe
y_pred_labels = label_encoder_genre.inverse_transform(y_pred)

# Add predictions to the test dataframe
test_df['top genre'] = y_pred_labels

# Print the predictions
print(test_df[['title', 'artist', 'top genre']].head(10))

# Save the predictions to a CSV file
output_file = 'stacking_predictions.csv'
test_df.to_csv(output_file, index=False)

print(f"Predictions saved to {output_file}")
```

7. Challenges and Solutions

Low Accuracy of Initial Models:

1. **Challenge**: The initial models showed limited accuracy, particularly struggling with genre differentiation.
   Solution: Introduced the 'artist' feature using One-Hot Encoding, which significantly improved classification by adding valuable categorical information.

Class Imbalance Issue:

2. **Challenge**: The dataset had imbalanced classes, where certain genres were underrepresented, leading to biased predictions.

Solution: Applied SMOTE (Synthetic Minority Over-sampling Technique) to balance the classes and improve model performance on minority genres.

8. Reference

- https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html
- https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html
- https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html
- https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.htm

etc